# Curso Tendencias Aplicadas de Ingenería de Software

Programación Asincrónica

# Programación asincrónica: concepto general

- La programación asincrónica es un modelo de ejecución donde las tareas no bloquean el flujo principal del programa mientras esperan la finalización de una operación.
- En lugar de ejecutar instrucciones de manera secuencial y esperar resultados inmediatos, el sistema puede continuar procesando otras tareas y manejar el resultado de la operación en un momento posterior.
- Este modelo es especialmente importante en sistemas distribuidos y aplicaciones modernas basadas en microservicios, donde las operaciones de entrada/salida (I/O), comunicación por red y acceso a bases de datos pueden introducir latencias significativas.

# Programación asincrónica: concepto general

- En lugar de pensar en un flujo lineal de ejecución, el sistema asincrónico se basa en eventos, callbacks, o mecanismos de mensajería que permiten desacoplar la producción de trabajo de su consumo.

- Desde la perspectiva de la Arq. de Software, este modelo permite mejorar atributos de calidad como:
  - Rendimiento
  - Escalabilidad
  - Responsividad

# Problema del modelo síncrono

- En un modelo síncrono tradicional, cada operación bloquea el hilo de ejecución hasta completar su tarea.
- Ejemplo típico:
  - Solicitud HTTP llega al servidor.
  - El servidor consulta base de datos.
  - El servidor espera respuesta.
  - El servidor responde al cliente.
- Problema:
  - Durante el tiempo de espera, el hilo queda bloqueado, reduciendo la capacidad del sistema para atender otras solicitudes.
- Esto genera:
  - Baja escalabilidad bajo carga.
  - Uso ineficiente de recursos.
  - Mayor latencia percibida.
- En sistemas modernos con alta concurrencia, este modelo se vuelve un cuello de botella significativo.

# Modelo asincrónico: idea fundamental

- En el modelo asincrónico, la operación se delega a un componente externo y el hilo principal continúa ejecutando otras tareas.
- Cuando el resultado está disponible, el sistema es notificado mediante un evento, callback o mensaje.
- Flujo conceptual:
  - Se inicia operación.
  - Se delega ejecución.
  - El sistema continúa operando.
  - Se recibe resultado posteriormente.
- Este modelo permite maximizar el uso del CPU y reducir tiempos de espera improductivos.
- En términos de Arq. de Software, este enfoque es fundamental en sistemas distribuidos donde la latencia de red es inevitable y debe ser gestionada mediante desacoplamiento.

# Rol del asincronismo en arquitectura de software

- El asincronismo no es solo una técnica de programación, sino un principio arquitectónico.
- Permite construir sistemas:
  - Desacoplados
  - Escalables
  - Basados en eventos

- En arquitecturas modernas, el asincronismo aparece en:
  - Microservicios
  - Sistemas event-driven
  - Colas de mensajes

- Su principal contribución es permitir que los componentes del sistema no dependan de la respuesta inmediata de otros componentes.

- Esto reduce el acoplamiento temporal entre servicios.

# Acoplamiento temporal y asincronismo

- Uno de los problemas clave en sistemas distribuidos es el acoplamiento temporal, que ocurre cuando dos componentes deben estar activos al mismo tiempo para comunicarse.
- Ejemplo síncrono:
  - Servicio A → espera respuesta de Servicio B
- Problema:
  - Si B falla o está lento, A también se degrada.
- En asincronismo:
  - Servicio A → envía mensaje → continúa ejecución
  - Servicio B procesa cuando puede
- Esto elimina la dependencia temporal estricta.
- Kleppmann destaca que el desacoplamiento temporal es una de las claves para construir sistemas altamente escalables.

# Mensajería como base del asincronismo

- En sistemas distribuidos, el asincronismo se implementa comúnmente mediante sistemas de mensajería.
- La mensajería es el paraguas general. Dentro de ella viven los patrones como:
  - Canales (queues)
  - Pub/sub
  - Etc
- Componentes:
  - Productores (generan mensajes)
  - Brokers (intermediarios)
  - Consumidores (procesan mensajes)
- Este modelo permite desacoplar completamente los servicios.
- Ejemplo:
  - Servicio de pagos → envía evento “pago realizado”
  - Servicio de notificaciones → procesa evento
  - Servicio de analítica → almacena evento
- Ningún servicio necesita conocer a los demás.

# Patrón Canales (Channels)

- El patrón de canales consiste en una comunicación asincrónica donde los mensajes se envían a través de un canal intermedio.
- Un canal es un modelo de mensajería donde los mensajes se envían a una cola compartida y son consumidos normalmente por un solo consumidor por mensaje.
- Un canal actúa como una cola o buffer entre productor y consumidor.
- Es como una fila de trabajo.
  - Productor -> [ COLA / CANAL ] -> Consumidor
- Propiedades clave
  - Cada mensaje es procesado por un solo consumidor
  - Se comporta como una cola (queue)
  - Orden generalmente FIFO (primero en entrar, primero en salir)

# Patrón Canales (Channels)

- Ejemplo real:  Sistema de procesamiento de pedidos:
  - Pedidos -> Canal -> Worker 1 / Worker 2 / Worker 3
- Cada pedido lo procesa solo un worker, no todos.
- Uso típico:
  - Procesamiento de tareas
  - Jobs en background
  - Colas de emails
- Características:
  - Comunicación indirecta
  - Desacoplamiento temporal
  - Control de flujo
  - Buffer de mensajes
- Ejemplo conceptual:
  - Producción -> Canal -> Consumo
- Los canales permiten absorber picos de carga sin sobrecargar los consumidores.

# Ejemplo conceptual de canales

- Imaginemos un sistema de procesamiento de pedidos:
- Usuario realiza compra.
  - Pedido se envía a un canal.
- Servicios independientes procesan:
  - Inventario
  - Facturación
  - Envío
- Cada servicio consume del canal a su propio ritmo.
- Ventaja:
  - El sistema puede manejar grandes volúmenes sin bloquear la aplicación principal.

# Publish / Subscribe (Pub/Sub)

- El patrón publish/subscribe es un modelo donde los productores publican eventos sin conocer a los consumidores.

- Pub/Sub es un modelo diferente donde un mensaje es enviado a múltiples consumidores interesados. Es como si fuese “un publicador de noticias”
- Propiedades clave
  - Un mensaje puede llegar a muchos consumidores
  - Los consumidores se “suscriben” a un tema (topic)
  - Productor no sabe quién consume
  - Alto desacoplamiento

# Publish / Subscribe (Pub/Sub)

- Componentes:
  - Publisher: emite eventos
  - Broker: distribuye eventos
  - Subscribers: reciben eventos

- A diferencia de los canales, múltiples consumidores pueden recibir el mismo evento.

# Diferencias entre canales y Pub/Sub

| **Característica** | **Canales** | **Pub/Sub** |
|---|---|---|
| Entrega | Un Consumidor | Multiplies consumidores |
| Modelo | Cola | Evento |
| Acoplamiento | Medio | Bajo |
| Uso | Procesamiento de tareas | Sistema de Eventos |

- Ejemplo de canal (Queue): un ticket de atención en un banco. Cada ticket lo atiende un solo empleado
  - Ticket 1 → Cajero A
  - Ticket 2 → Cajero B

# Diferencias entre canales y Pub/Sub

- Pub/Sub: un canal de YouTube. Se sube un video (evento)
  - Todos los suscriptores lo reciben
  - Video nuevo -> todos los suscriptores

# Asincronismo en microservicios

- En arquitecturas de microservicios, el asincronismo permite:
  - Evitar dependencias directas entre servicios
  - Mejorar tolerancia a fallos
  - Escalar servicios de forma independiente
- Ejemplo:
  - Servicio de pedidos no espera respuesta del servicio de envío, sino que emite un evento.
- Esto permite que el sistema continúe funcionando incluso si un servicio está temporalmente caído